Introduction to Julia : GPU programming

Matteo Foglieni, Dr. Martin Ohlerich

GPU: NVIDIA A100 (GA100)

- GPC ×7 — Graphics Processing Cluster
  - TPC ×7-8/GPC — Texture Processing Cluster
    - SM ×2/TPC, 108 total — **Streaming Multiprocessor (SMs)**
      - Sub-partition ×4
        - Warp Scheduler, 32 threads/clk
        - Dispatch Unit, 32 threads/clk
        - Register File, 16,384 × 32-bit
        - **FP32** ×16/sub -> **64/SM**, aka **"CUDA cores" or "Shading Units"**
        - INT32 ×16/sub -> 64/SM
        - FP64 ×8/sub -> 32/SM, 1:2 FP64:FP32, HPC grade
        - **Tensor Core** ×1/sub -> **4/SM**
        - LD/ST ×8/sub -> 32/SM, load/store units
        - SFU ×1/sub -> 4/SM, sin/cos/exp/rsqrt
      - L1 Data Cache / Shared Memory 192KB/SM
      - Tex (TMU) ×4 -> 4/SM

Total: **6912 Shading Units, 432 Tensor Cores**

Sources: NVIDIA A100 Tensor Core GPU Architecture, page 22, and TechPowerUp page for A100 GPU

GPU terminology

PROGRAM ABSTRACTIONS

Grid
NDRange

Thread Block (TB)
WorkGroup (WG)

CUDA thread
Work Item

32/64 ≤ #threads ≤ 1024

practical efficiency minimum
(= 1 warp), not a constraint

warp scheduler **hardware constraint:**
it can track at most 32 warps per block
each warp is 32 threads
32*32 = 1024

TO BE CHOSEN:
#Thread Blocks
#threads in each TB

one core executes
instructions from one thread

Warp
Wavefront

32 threads

typical unit of execution on a GPU

all threads in a warp are scheduled together and execute in parallel onto a single SM

GPU HARDWARE

GPU

~10-1000 SM

Streaming Multiprocessor (SM)
Compute Unit (CU)

CORE CORE
CORE CORE
CORE CORE

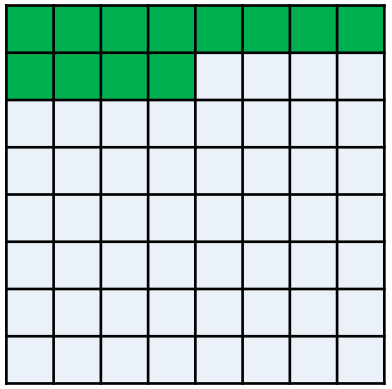
~64 CUDA cores

GPU terminology

E.g.: #threads in each Thread Block = 64 = 8x8

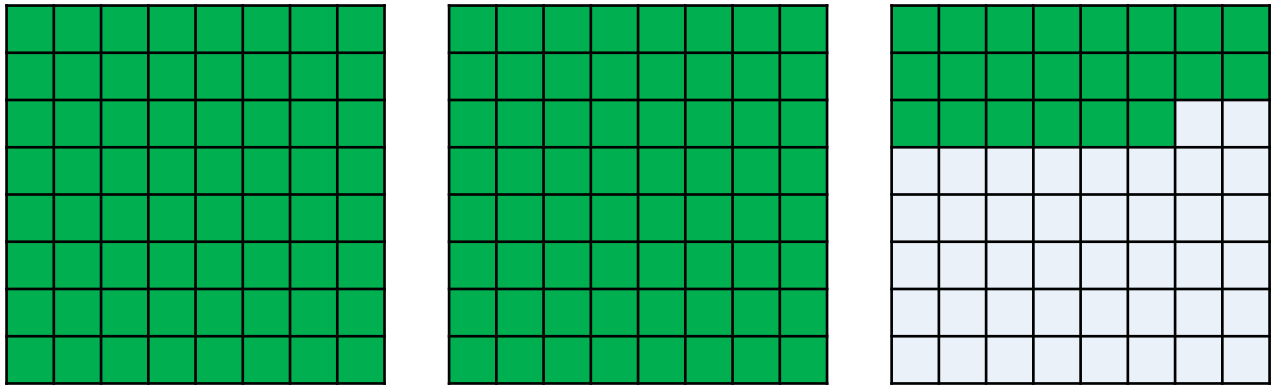
E.g.: Vector of length 12

1 TB is enough



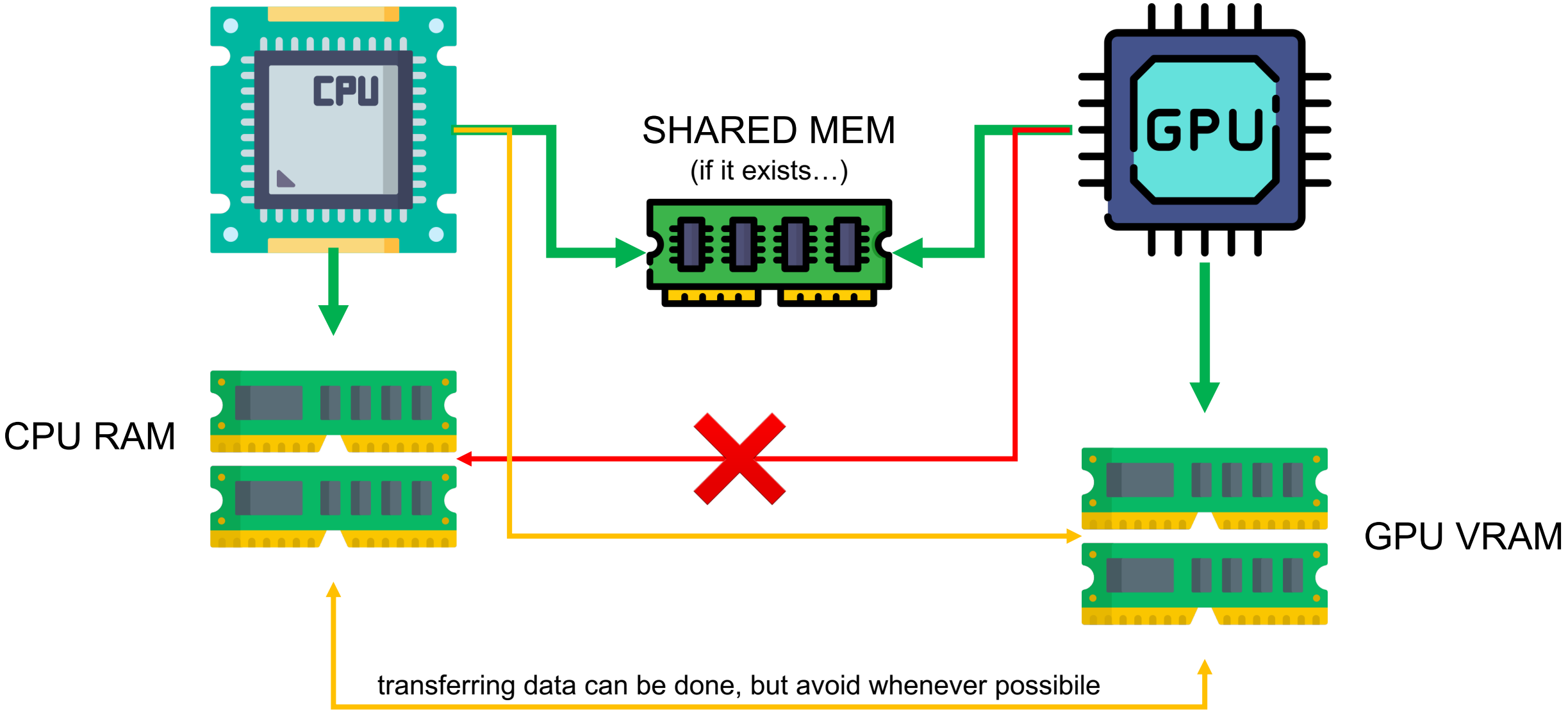
E.g.: Vector of length 150

We need 3 TBs (at least)

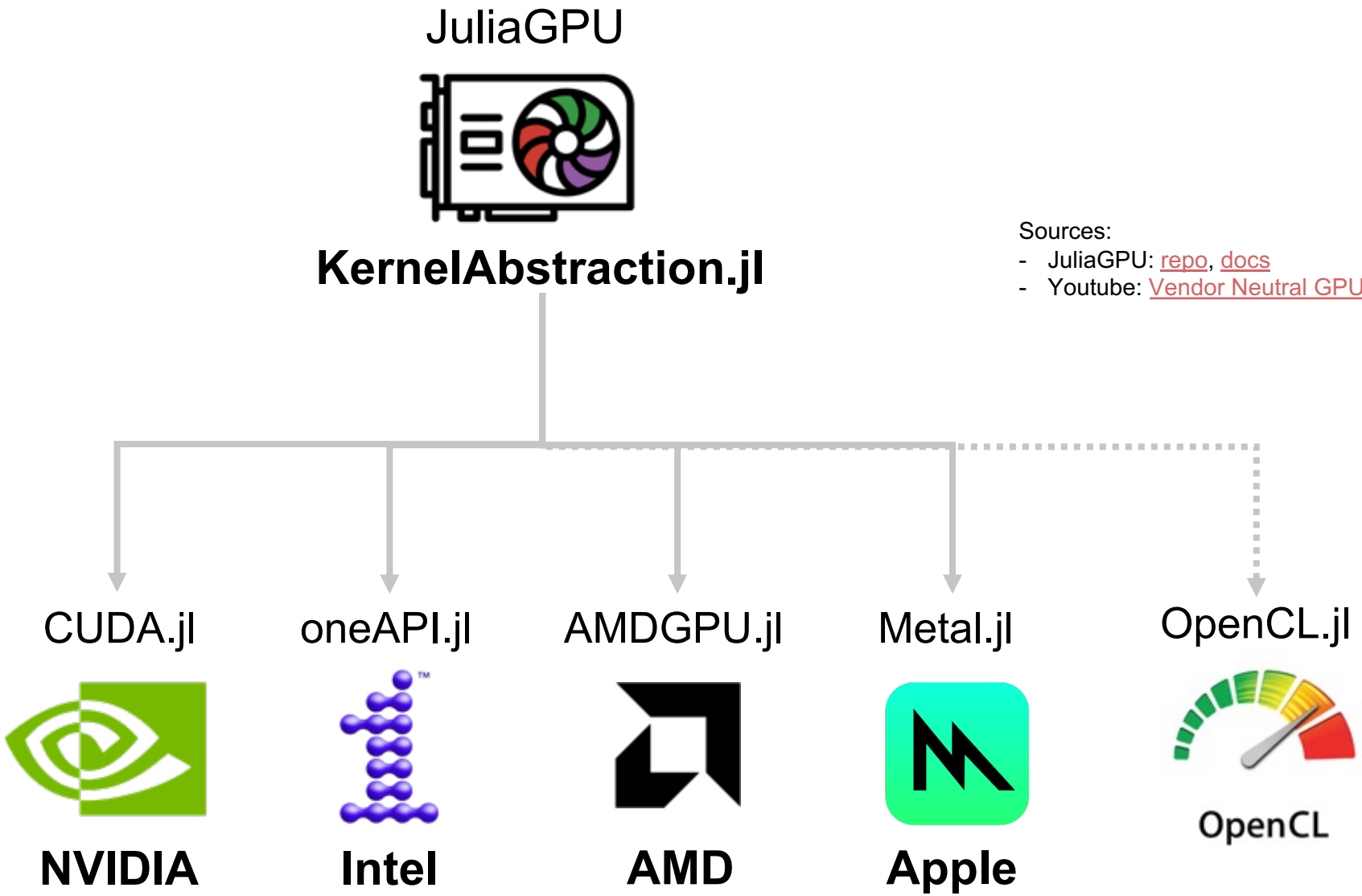


64 + 64 + 22

GPU terminology



JuliaGPU



Sources:
- JuliaGPU: repo, docs
- Youtube: Vendor Neutral GPU Programming in Julia

KernelAbstractions.jl

KA example

```
1   using KernelAbstractions, Random, Test
2   using Metal
3   const DevLibrary = Metal
4   const backend = Metal.MetalBackend()
5   const DevArray = Metal.MtlArray
6   const DevFloat = Float32
7
8   @kernel function add_arrays!(a, b, c)
9       i = @index(Global)
10      c[i] = a[i] + b[i]
11  end
12
13  N = 1024
14  dev_a = rand!(allocate(backend, DevFloat, N))
15  dev_b = rand!(allocate(backend, DevFloat, N))
16  dev_c = similar(dev_a)
17
18  # Compile the kernel for this backend
19  compiled_kernel! = add_arrays!(backend, 64)
20  # Launch the kernel
21  event = compiled_kernel!(dev_a, dev_b, dev_c; ndrange=size(dev_a))
22  KernelAbstractions.synchronize(backend)
23
24  a, b, c = Array(dev_a), Array(dev_b), Array(dev_c)
25  @test all([z ≈ x + y for (x,y,z) in zip(a,b,c)])
```

only part to be customized!

```
julia> include("./KA-outline_Metal.jl")
Test Passed
```

WorkGroup (Thread Block) size; use 32 or 64

NDRange (Grid) size; set to vector size

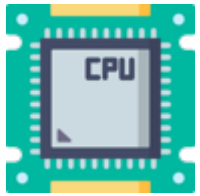
Backends

```
using CUDA
const DevLibrary = CUDA
const backend = CUDA.CUDAKernels.CUDABackend()
const DevArray = CUDA.CuArray
const DevFloat = Float64
```

```
using oneAPI
const DevLibrary = oneAPI
const backend = oneAPI.oneAPIKernels.oneAPIBackend()
const DevArray = oneAPI.OneArray
const DevFloat = Float64
```

```
using AMDGPU
const DevLibrary = AMDGPU
const backend = AMDGPU.ROCKernels.ROCBackend()
const DevArray = AMDGPU.ROCArray
const DevFloat = Float64
```

```
using Metal
const DevLibrary = Metal
const backend = Metal.MetalBackend()
const DevArray = Metal.MtlArray
const DevFloat = Float32
```




```
const backend = CPU()
const DevArray = Array
const DevFloat = Float64
```

Backend.jl

Why not using

```
if Base.find_package("oneAPI") !== nothing
```

?

We can unify the previous code snippets into a single Backend.jl file!

Portability ✅

```
using Adapt, KernelAbstractions

onlycpu = (@isdefined onlycpu ) ? onlycpu : false
if Base.find_package("oneAPI") !== nothing && onlycpu == false
    using oneAPI
    const backend = oneAPI.oneAPIKernels.oneAPIBackend()
    const DevArray = oneAPI.oneArray
    const DevFloat = Float64
    println("Using oneAPI.jl")
    oneAPI.versioninfo()

elseif Base.find_package("CUDA") !== nothing && onlycpu == false
    using CUDA
    const backend = CUDA.CUDAKernels.CUDABackend()
    const DevArray = CuArray
    const DevFloat = Float64
    CUDA.allowscalar(false)
    println("Using CUDA.jl")
    CUDA.versioninfo()

elseif Base.find_package("AMDGPU") !== nothing && onlycpu == false
    using AMDGPU
    const backend = AMDGPU.ROCKernels.ROCBackend()
    const DevArray = ROCArray
    const DevFloat = Float64
    println("Using AMDGPU.jl")
    AMDGPU.versioninfo()

elseif Base.find_package("Metal") !== nothing && onlycpu == false
    using Metal
    const backend = Metal.MetalKernels.MetalBackend()
    const DevArray = Metal.MtlArray
    const DevFloat = Float32
    println("Using Metal.jl => Float32")
    Metal.versioninfo()

else
    using ThreadPinning
    const backend = CPU()
    const DevArray = Array
    const DevFloat = Float64
    println("Using CPU and ThreadPinning.jl")
    pinthreads(:cores)
    threadinfo()
end
```

Common API of Backends

Example: use versioninfo()

Reproducibility ✅

```
julia> versioninfo()
Julia Version 1.10.9
Commit 5595d20a287 (2025-03-10 12:51 UTC)
Build Info:
  Official https://julialang.org/ release
Platform Info:
  OS: macOS (arm64-apple-darwin24.0.0)
  CPU: 10 × Apple M1 Pro
  WORD_SIZE: 64
  LIBM: libopenlibm
  LLVM: libLLVM-15.0.7 (ORCJIT, apple-m1)
Threads: 4 default, 0 interactive, 2 GC (on 8 virtual cores)
Environment:
  JULIA_NUM_THREADS = 4
```

```
julia> oneAPI.versioninfo()
Binary dependencies:
- NEO: 24.26.30049+0
- libigc: 1.0.17193+0
- gmmlib: 22.3.20+0
- SPIRV_LLVM_Translator: 20.1.0+2
- SPIRV_Tools: 2025.1.0+1

Toolchain:
- Julia: 1.10.9
- LLVM: 15.0.7

1 driver:
- 00000000-0000-0000-1847-b9f301037561 (v1.3.30049, API v1.3.0)

8 devices:
- Intel(R) Data Center GPU Max 1550
- Intel(R) Data Center GPU Max 1550
- Intel(R) Data Center GPU Max 1550
- Intel(R) Data Center GPU Max 1550
- Intel(R) Data Center GPU Max 1550
- Intel(R) Data Center GPU Max 1550
- Intel(R) Data Center GPU Max 1550
- Intel(R) Data Center GPU Max 1550
```

NOTE: 2 tiles per GPU

Example of Adapt custom structs in KA

Using struct in kernel

We need to import a struct as a params in the kernel

```
1   include("./Backend.jl")
2   using Test
3
4   struct MyStruct
5       v::Vector{Float32}
6       f::Float32
7   end
8
9   MS = MyStruct([1.0f0, 2.0f0], 3.0f0)
10
11  @kernel function simple_kernel!(B, A, @Const(ms))
12      i = @index(Global)
13      B[i] = ms.f * A[i] + ms.v[2]
14  end
15
16  dev_A = KernelAbstractions.ones(backend, DevFloat, 512, 512)
17  dev_B = similar(dev_A)
18  compiled_kernel! = simple_kernel!(backend, 64)
19  compiled_kernel!(dev_B, dev_A, MS, ndrange=size(dev_A))
20  KernelAbstractions.synchronize(backend)
21
22  B = Array(dev_B)
23  @test all(B .≈ DevFloat(5.0))
```

```
julia> include("./GaPSE-KA_error-isbits-1.jl")
Using Metal.jl => Float32
ERROR: LoadError: GPU compilation of MethodInstance for
gpu_simple_kernel!(::KernelAbstractions.CompilerMetadata
{…}, ::MtlDeviceMatrix{…}, ::MtlDeviceMatrix{…}, ::MyStr
uct) failed
KernelError: passing non-bitstype argument

Argument 5 to your kernel function is of type MyStruct,
which is not a bitstype:
  .v is of type Vector{Float32} which is not isbits.
```

Using struct in kernel

maybe if I use the Device Array…

```julia
include("./Backend.jl")
using Test

struct MyStruct
    v::DevArray{Float32}
    f::Float32
end

MS = MyStruct(DevArray([1.0f0, 2.0f0]), 3.0f0)

@kernel function simple_kernel!(B, A, @Const(ms))
    i = @index(Global)
    B[i] = ms.f * A[i] + ms.v[2]
end

dev_A = KernelAbstractions.ones(backend, DevFloat, 512, 512)
dev_B = similar(dev_A)
compiled_kernel! = simple_kernel!(backend, 64)
compiled_kernel!(dev_B, dev_A, MS, ndrange=size(dev_A))
KernelAbstractions.synchronize(backend)

B = Array(dev_B)
@test all(B .≈ DevFloat(5.0))
```

```
julia> include("./GaPSE-KA_error-isbits-2.jl")
Using Metal.jl => Float32
ERROR: LoadError: GPU compilation of MethodInstance for
gpu_simple_kernel!(::KernelAbstractions.CompilerMetadata
{…}, ::MtlDeviceMatrix{…}, ::MtlDeviceMatrix{…}, ::MyStr
uct) failed
KernelError: passing non-bitstype argument

Argument 5 to your kernel function is of type MyStruct,
which is not a bitstype:
```

Using struct in kernel

```julia
include("./Backend.jl")
using Test

struct MyStruct{V,F}
    v::V    # Vector{Float32}
    f::F    # Float32
end

function Adapt.adapt_structure(to, ms::MyStruct)
    MyStruct(
        adapt(to, ms.v),
        ms.f
    )
end

#Adapt.@adapt_structure MyStruct    # easier, but not always possible
MS = MyStruct(DevArray([1.0f0, 2.0f0]), 3.0f0)

@kernel function simple_kernel!(B, A, @Const(ms))
    i = @index(Global)
    B[i] = ms.f * A[i] + ms.v[2]
end

dev_A = KernelAbstractions.ones(backend, DevFloat, 512, 512)    # alloc on device
dev_B = similar(dev_A)
compiled_kernel! = simple_kernel!(backend, 64)
compiled_kernel!(dev_B, dev_A, MS, ndrange=size(dev_A))
KernelAbstractions.synchronize(backend)

B = Array(dev_B)
@test all(B .≈ DevFloat(5.0))

```

Parametric Struct + Adapt.jl

https://cuda.juliagpu.org/stable/tutorials/custom_structs

```
julia> include("./GaPSE-KA_working.jl")
Using Metal.jl => Float32
Test Passed
```

KA.jl and GaPSE.jl

For GaPSE.jl we did something more

```
82  struct MySpline
83      xs::Vector{DevFloat}
84      coeffs::Vector{Tuple{DevFloat,DevFloat,DevFloat,DevFloat}}
85      N::Int64
86      function MySpline(xs, ys; bc="Error", ic="Natural")
87          #...    # cubic spline computation
88          new(xs, coeffs, N)
89      end
90  end
91
92
93  function (S::MySpline)(x)
94      @assert S.xs[1] ≤ x ≤ S.xs[end] "BC Error"
95      i = searchsortedlast(S.xs, x)
96      u, a, b, c, d = (i == length(S.xs)) ?
97          (x - S.xs[i-1], S.coeffs[i-1]...) :
98          (x - S.xs[i], S.coeffs[i]...)
99      (u ≈ 0.0) && (return S.coeffs[i][1])
100
101     return a + b * u + c * u^2 + d * u^3
102 end
```

Original Spline implementation

```
106 struct DevMySpline{V,VT,I}
107     xs::V       #Vector{DevFloat}
108     coeffs::VT  #Vector{Tuple{DevFloat,DevFloat,DevFloat,DevFloat}}
109     N::I        #Int64
110 end
111
112 function Adapt.adapt_structure(to, s::MySpline)
113     return DevMySpline(
114         adapt(to, s.xs),
115         adapt(to, s.coeffs),
116         adapt(to, s.N))
117 end
118
119 function Adapt.adapt_structure(to, s::DevMySpline)
120     DevMySpline(
121         adapt(to, s.xs),
122         adapt(to, s.coeffs),
123         adapt(to, s.N))
124 end
125
126
127 function (S::DevMySpline)(x)
128     i = searchsortedlast(S.xs, x)
129     u, a, b, c, d = (i == length(S.xs)) ?
130         (x - S.xs[i-1], S.coeffs[i-1]...) :
131         (x - S.xs[i], S.coeffs[i]...)
132     (u ≈ zero(u)) && (return S.coeffs[i][1])
133
134     return a + b * u + c * u^2 + d * u^3
135 end
```

Code added

KA.jl and GaPSE.jl

Example of usage of a MySpline in a kernel:

```
138  xs = [1.0f0 * i for i in 1:20]
139  ys = xs.^2
140  MS = MySpline(xs, ys)
141
142  xs_dev = DevArray(xs)
143  ys_dev = DevArray(ys)
144  MS_dev = adapt(backend, MS)
145
146  @kernel function spline_kernel!(b, a, @Const(ms))
147      i = @index(Global, Linear)
148      b[i] = ms(a[i])
149  end
150
151  a = [DevFloat(MS.xs[begin] + rand() * (MS.xs[end] - MS.xs[begin])) for i in 1:1024]
152  a_dev = DevArray(a)
153  b_dev = similar(a_dev)
154
155  spline_kernel!(backend, 64)(b_dev, a_dev, MS_dev, ndrange=size(a_dev))
156  KernelAbstractions.synchronize(backend)
157
158  b = Array(b_dev)
159
160  @test all(b.≈MS.(a))
```

conversion with Adapt.jl

```
julia> include("./GaPSE-KA_devstruct.jl")
Using Metal.jl => Float32
Test Passed
```

Appendices

Julia vs C++ GEMM on LRZ hardware (CPU only)

GEMM benchmarking comparison on LRZ Hardware

we chose to compare
a simple matrix multiplication

- blas_gemm and blas_gemm-usm with icx
- Julia with LinearAlgebra

$$R = \alpha A * B + \beta C \qquad \alpha, \beta \in \mathbb{R}$$

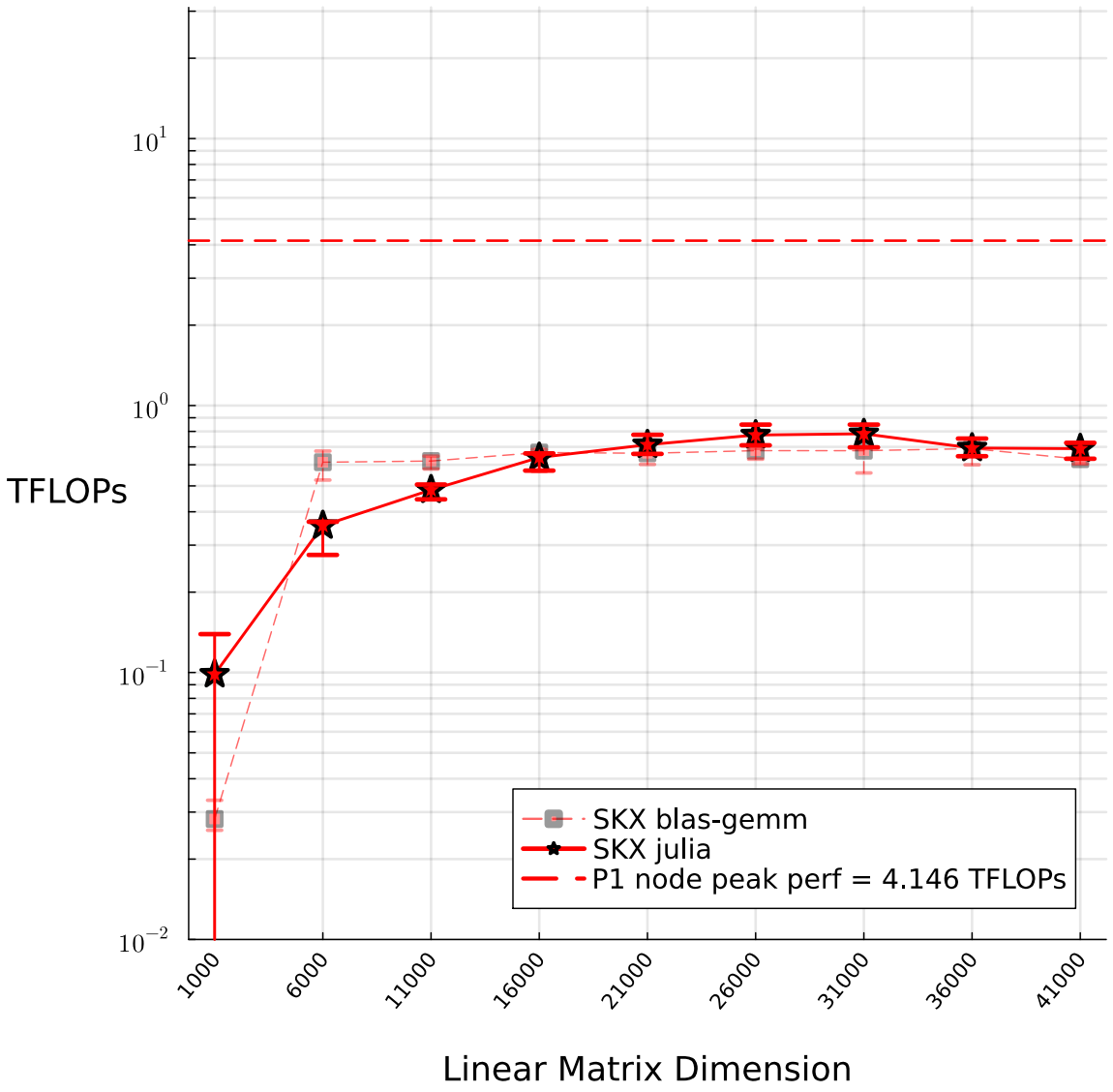
$A: M \times N$, $B: N \times K$, $C: M \times K$

MLD = Matrix Linear Dimension

Our simplification: $M = N = K = \mathbf{MLD} \qquad \alpha = 1, \beta = 0$

$$\mathbf{FLOPs} = \frac{\#\,\text{FP ops}}{\text{time}} \approx \frac{M \times N \times K}{\Delta t} = \frac{\mathbf{MLD}^3}{\Delta t}$$

GEMM benchmarking comparison on LRZ hardware



SKX = Intel “Skylane” Xeon Platinum 8174

SPR = Intel “Sapphire Rapid” Xeon Platinum 8480L

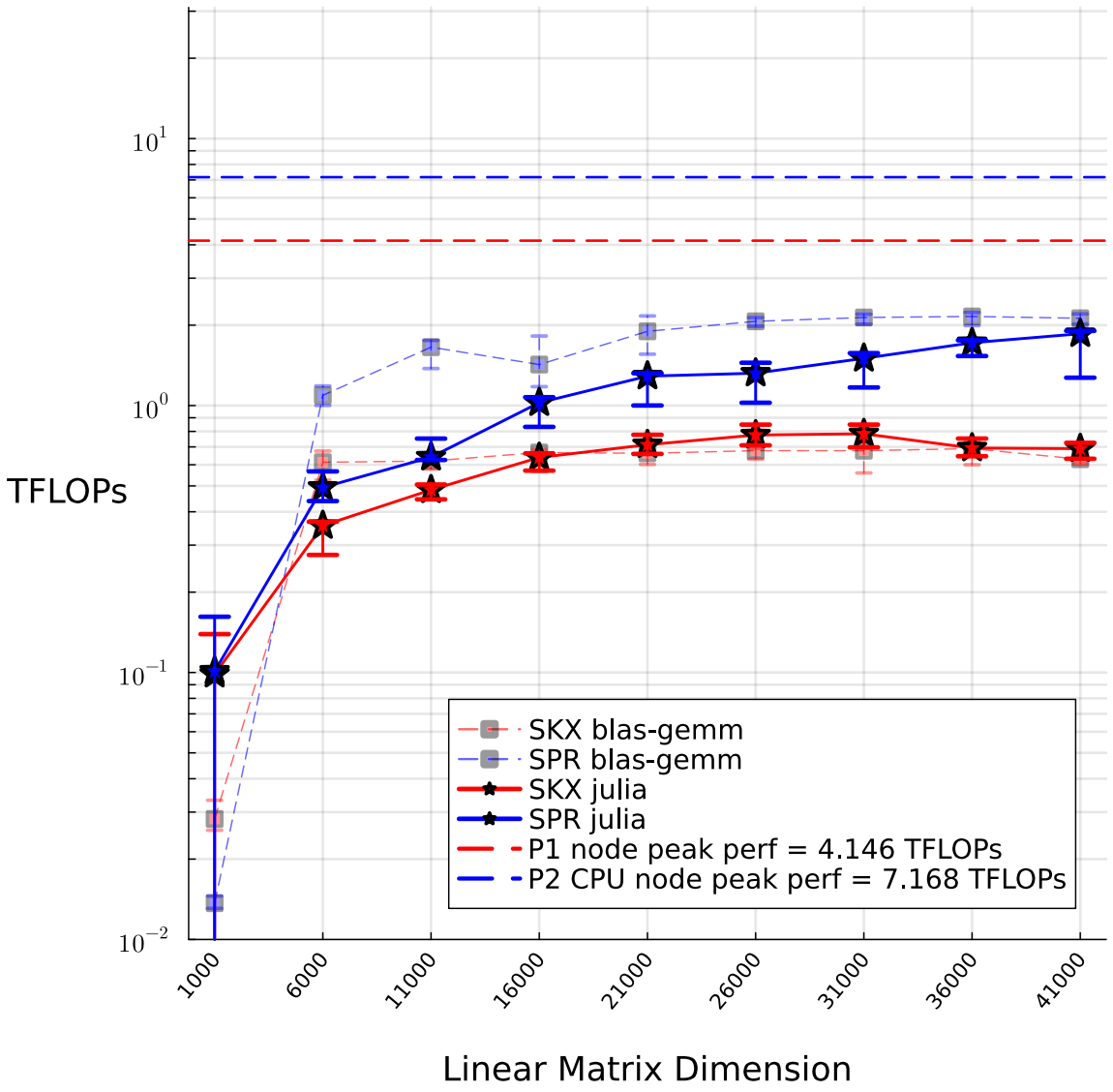
PVC = Intel “Ponte Vecchio” Data Center GPU Max 1550

P1 = SuperMUC-NG Phase 1
(each node has 2 SKX CPUs)

P2 = SuperMUC-NG Phase 2
(each node has 2 SPR CPUs and 4 PVC GPUs)

higher is better

GEMM benchmarking comparison on LRZ hardware



SKX = Intel “Skylane” Xeon Platinum 8174

SPR = Intel “Sapphire Rapid” Xeon Platinum 8480L

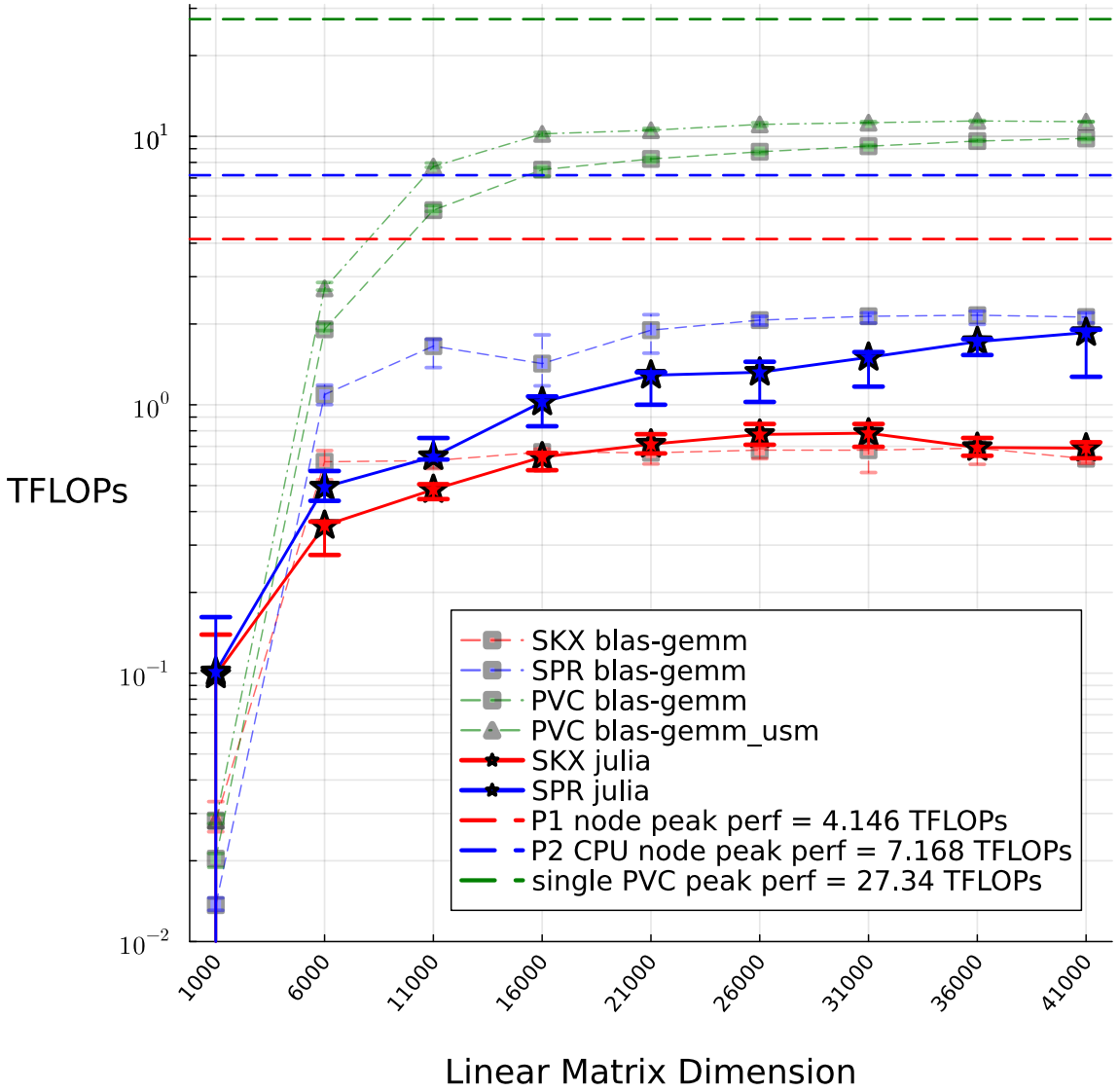
PVC = Intel “Ponte Vecchio” Data Center GPU Max 1550

P1 = SuperMUC-NG Phase 1
(each node has 2 SKX CPUs)

P2 = SuperMUC-NG Phase 2
(each node has 2 SPR CPUs and 4 PVC GPUs)

higher is better

GEMM benchmarking comparison on LRZ hardware



SKX = Intel “Skylane” Xeon Platinum 8174

SPR = Intel “Sapphire Rapid” Xeon Platinum 8480L

PVC = Intel “Ponte Vecchio” Data Center GPU Max 1550

P1 = SuperMUC-NG Phase 1
(each node has 2 SKX CPUs)

P2 = SuperMUC-NG Phase 2
(each node has 2 SPR CPUs and 4 PVC GPUs)

higher is better